

User Guide

For PPG Mobile App

Table of Contents

1.System architecture	3
2.Supplied files and Resources	3
3.Requirements	4
4.Install the app on the phone	4
5.First launch.....	5
6.Pair the phone and laptop.....	5
6.1 Windows	5
6.2 Linux	6
7.Streaming from a Windows laptop	6
8.Streaming from a Linux laptop	7
9.Reading the display.....	7
9.1 Waveforms	8
9.2 Parameter chips	8
9.3 RR method	8
9.4 "--" is a value, not a fault	9
10.Process Termination and Restart.....	9
11.Recordings and sample rate	9
12.Troubleshooting	9
Appendix A — Command reference	10
A.1 Windows	10
A.2 Linux	10
A.3 Installing the app, if adb is available.....	10

PPG Mobile App — User Guide

Version: 1.0

Scope: The PPG Monitor Android application, and how to stream a recording to it over Bluetooth from a laptop.

Audience: Anyone who needs to watch a PPG recording being analysed on an Android phone.

Licence: Apache-2.0. Copyright 2026 Prajnaana Technologies Pvt Ltd.

This guide covers installation and operation. The signal-processing method is described in the technical document.

PPG Monitor is an Android application that analyses a photoplethysmogram on the phone and displays live heart rate, breathing rate, heart-rate variability, and the PPG waveform.

Samples are transmitted to the phone over **Bluetooth** from a laptop, which replays a recording or a live sensor feed.

1. System architecture

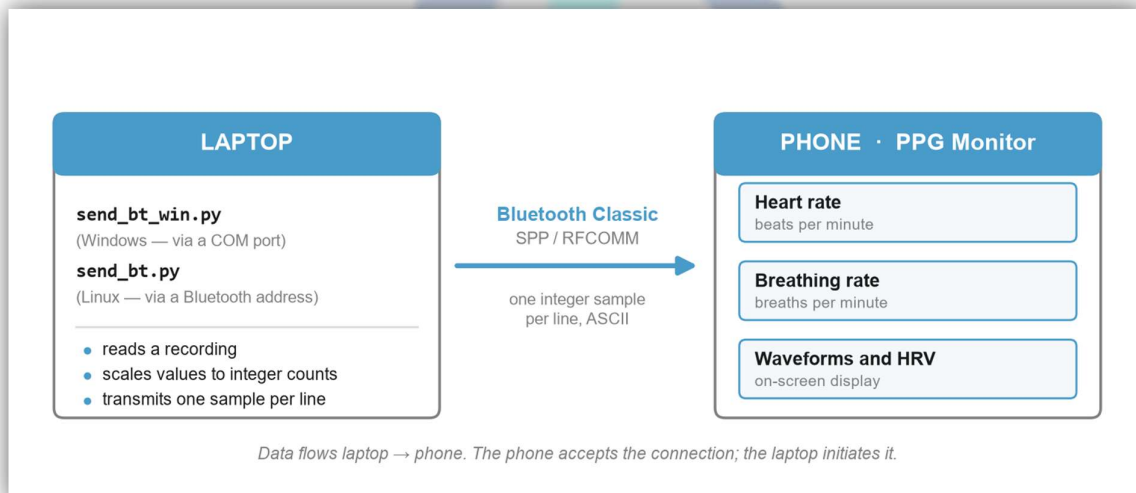


Figure 1 — The laptop transmits a recording over Bluetooth; the phone receives the samples and displays the results.

The transmitter is the laptop script, not an application.

The receiver is the phone application. It displays the results as the samples arrive.

Data flows **laptop** → **phone**. The phone accepts the connection; the laptop initiates it.

2. Supplied files and Resources

File	Purpose
app-debug.apk	The Android application
send_bt_win.py	Sender for Windows laptops
send_bt.py	Sender for Linux laptops
synthesized_sample_adult.txt & synthesized_sample_neonate.txt	Sample recording — adult/neonate, 125 Hz

Each sender is a single self-contained file. The scale-aware recording reader is built into both, so a sender can be copied anywhere on its own and needs no other supplied Python file beside it.

A recording is also required — either the supplied sample, or any plain-text file holding one value per line.

3. Requirements

Phone

Android 7.0 (API 24) or later. The application contains native libraries for arm64-v8a, armeabi-v7a, x86 and x86_64, so all current devices are supported.

Bluetooth Classic. A **physical device is required** — Bluetooth Classic is not available on the Android emulator.

Laptop

Windows or Linux, with a working Bluetooth Classic radio.

Python 3.8 or later.

One Bluetooth package, according to platform:

Platform	Package	Install
Windows	pyserial	pip install pyserial
Linux	pybluez2	sudo apt-get install -y libbluetooth-dev then pip install pybluez2

Windows requires no Bluetooth library beyond pyserial, because the phone's serial service is reached through a virtual COM port.

4. Install the app on the phone

The application may be installed by either method.

a) By copying the file to the phone. Transfer app-debug.apk by email, cloud storage, or USB, then tap it in the phone's file manager. Permission to install from unknown sources must be granted when prompted, followed by **Install**.

b) Via adb, if the Android platform-tools are available and USB debugging is enabled on the phone:

```
adb install -r -t app-debug.apk
```

If more than one device is attached, the target must be named explicitly:

```
adb devices
```

```
adb -s <serial> install -r -t app-debug.apk
```

Installing over an existing copy that was signed with a different key fails with `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. Removing the earlier copy resolves it — either uninstall it on the phone, or run `adb uninstall com.example.ppgmonitor`.

5. First launch

Open **PPG Monitor** on the phone. On first launch the app requests the **Nearby devices** (Bluetooth) permission; this must be allowed, otherwise the app cannot listen for a connection.

The screen then shows *"Waiting for laptop to connect over Bluetooth..."*.

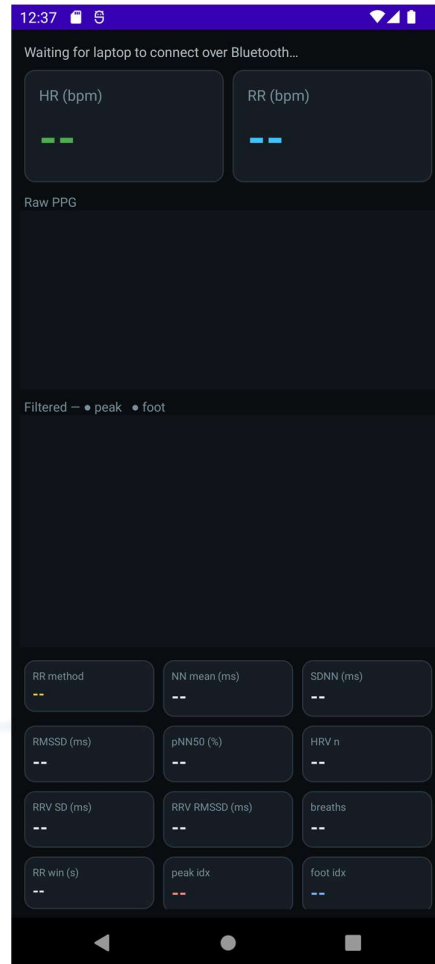


Figure 2 — The app after launch, before a sender has connected. Every reading shows -- and both waveform panes are empty. This is the expected idle state, not a fault.

The app must be **open and, in the foreground**, whenever a sender is started. The phone advertises its Bluetooth serial service only while the app is running, so while the app is closed there is nothing for the laptop to connect to.

6. Pair the phone and laptop

This is required once per phone.

The phone's Bluetooth address, useful for identifying it in the device list, is shown at **Settings** → **About phone** → **Status** → **Bluetooth address**.

6.1 Windows

Settings → **Bluetooth & devices** → **Add device** → select the phone → confirm the code on both sides.

6.2 Linux

```
bluetoothctl
power on
scan on
pair XX:XX:XX:XX:XX:XX
trust XX:XX:XX:XX:XX:XX
scan off
quit
```

The pairing prompt must be accepted on the phone. Substitute the phone's own Bluetooth address for the one shown.

7. Streaming from a Windows laptop

Windows exposes the phone's Bluetooth serial service as a **virtual COM port**, which `send_bt_win.py` opens. The port must be created before the sender will run, and it can be created only while the app is advertising the service.

Step 1 — open the app on the phone and wait for *"Waiting for laptop to connect over Bluetooth..."*.

Step 2 — add an outgoing COM port. This is required once per phone. Open the classic Bluetooth Settings dialog by either route:

Settings → Bluetooth & devices → Devices → scroll to Related settings → More Bluetooth settings, or **Win + R**, then:

```
rundll32.exe shell32.dll,Control_RunDLL bthprops.cpl,,1
```

In that dialog: **COM Ports** tab → **Add...** → **Outgoing** → **Browse...** → select the phone → select its SPP service → **OK**. Windows assigns a port, for example COM3.

The assigned port can be confirmed from PowerShell:

```
[System.IO.Ports.SerialPort]::GetPortNames()
```

No output means no port exists and step 2 has not completed. Once created, the port persists across reboots and need not be added again.

Step 3 — stream the recording:

```
pip install pyserial
python send_bt_win.py COM3 ..\synthesized_sample_adult.txt 125 --subject adult
python send_bt_win.py COM3 ..\synthesized_sample_neonate.txt 125 --subject neonate
```

Arguments: ``<COM port> <recording file> [rate_hz]``. The rate defaults to 125.

The COM port comes **first**. Supplying the recording where the port is expected is the commonest mistake; it fails inside pyserial with `SerialException: Cannot configure port ... OSError(22, 'The parameter is incorrect.')`, because the filename has been interpreted as a serial device.

Opening the port is what initiates the Bluetooth connection, so a brief delay before connected; streaming appears is normal. The phone's status line changes to *"Connected: ..."* and a sample counter climbs in the console:

```
opening COM3 (this initiates the Bluetooth connection) ...
connected; streaming (Ctrl-C to stop)
Input scale: --scale 1000000 (auto: decimal text, 8 fractional digits, ...)
sent 125 samples (1s)
sent 250 samples (2s)
```

8. Streaming from a Linux laptop

send_bt.py connects directly by Bluetooth address, so no COM port is involved.

Step 1 — open the app on the phone, as in section 7.

Step 2 — install the Bluetooth package, once:

```
sudo apt-get install -y libbluetooth-dev
pip install pybluez2
```

Step 3 — stream the recording:

```
python3 send_bt.py xx:xx:xx:xx:xx:xx ../synthesized_sample_adult.txt 125 --subject adult
python3 send_bt.py xx:xx:xx:xx:xx:xx ../synthesized_sample_neonate.txt 125 --subject neonate
```

Arguments: <phone Bluetooth address> <recording file> [rate_hz].

The script lists the SPP services it discovers, connects to the one named **PPGMonitor**, and streams.

Connecting by Bluetooth address is **not** available on Windows. PyBluez does not build on current Windows Python toolchains, and Python's socket module provides no AF_BTH support there. The COM port created in section 7 is the Windows equivalent — it is bound to the phone's address by Windows itself.

9. Reading the display

HR (bpm) and **RR (bpm)** are shown as large cards pinned at the top; they remain visible while the rest of the screen scrolls. Below them are two waveforms and a grid of parameter chips.



Figure 3 — The display while a recording is streaming. Both waveforms are drawing, red and blue markers sit on the detected peaks and feet, and the heart-rate-variability grid has populated.

9.1 Waveforms

Raw PPG (grey) — the raw sample as received.

Filtered (green) — the processed signal, with markers:

- **red** = systolic **peak**
- **blue** = pulse onset (**foot**)

Markers are positioned at the sample where the peak or foot actually occurred, so they align correctly with the waveform. A marker may appear a moment after the beat itself. This is expected.

9.2 Parameter chips

Chip	Meaning
HR (bpm)	Heart rate (requires ≥ 2 beats)
RR (bpm)	Respiratory rate (requires a full respiratory window)
RR method	Confidence of the respiratory-rate result — see below
NN mean (ms)	Mean of normal-to-normal (beat) intervals
SDNN (ms)	Standard deviation of NN intervals
RMSSD (ms)	Root-mean-square of successive NN differences
pNN50 (%)	Percentage of successive NN differences > 50 ms
HRV n	Number of intervals the HRV figures rest on
RRV SD / RMSSD (ms)	Respiratory-rate variability
breaths	Breath intervals behind the RRV figures
RR win (s)	Respiratory analysis window: reached / target
peak idx / foot idx	Sample index of the last detected peak / foot
samples	Total samples processed

9.3 RR method

The respiratory rate is derived from three independent estimates. The label reports how closely they agree:

Code	Label
ALL_THREE	Strong match
TWO_AGREE	Moderate match
DECLINED	Weak match
REJECTED	No match

A trailing ... (for example "Strong match...") indicates a **provisional** verdict — the analysis window is still filling.

9.4 "--" is a value, not a fault

Every rate requires history before it carries meaning: two beats for HR, a full respiratory window (8–65 s depending on patient type) for RR, and several intervals for HRV. Until then the field reads --. It must not be read as zero.

10. Process Termination and Restart

Ctrl+C stops the sender. The app re-arms its listener automatically, so a sender may be stopped and re-run at any time **without** restarting the app, re-pairing, or recreating the COM port. Each run begins from a clean state, so results from a previous run do not carry over.

11. Recordings and sample rate

Recordings are plain text, one value per line. Decimal recordings are converted to integer counts automatically by the sender, which reports the multiplier it selected, for example:

Input scale: --scale 1000000 (auto: decimal text, 8 fractional digits, largest |value| 1 over 2000 samples)

Any recording may be streamed by passing its path to the sender in place of synthesized_sample_adult.txt.

The supplied application is configured for 125 Hz, adult. The rate argument given to the sender controls only the *speed* of transmission; it does not change the application's configuration. Streaming a recording made at a different rate therefore produces plausible but incorrect readings rather than an obvious failure. A version configured for other values must be requested.

12. Troubleshooting

- **`cannot find the recording ...`.** The named file is not where the sender looked, and the message lists every place it tried. A plain filename is searched for in the current folder, beside the sender script, and in the folder above it — so the supplied samples are found by name from either PPG_Android_App/ or PPG_Android_App/python_sources/. Any other location must be given as a path.
- **`ModuleNotFoundError: No module named 'serial'` (Windows).** Run pip install pyserial.
- **`ModuleNotFoundError: No module named 'bluetooth'` (Linux).** Run pip install pybluez2, preceded by sudo apt-get install -y libbluetooth-dev.
- **`SerialException: Cannot configure port ... The parameter is incorrect` (Windows).** The first argument was not a COM port. The order is send_bt_win.py <COM port> <recording file> [rate_hz]. If the order is correct, the port may not exist — confirm with [System.IO.Ports.SerialPort]::GetPortNames().
- **The Add → Outgoing browse list is empty (Windows).** The phone is not paired, or the app is not running on it. Both are required before a serial service is advertised.
- **`No SPP service found` / `PPGMonitor service not found` (Linux).** The app must be open and, in the foreground, showing "Waiting...", and the devices must be paired.
- **The app shows "Waiting..." and never connects, while the sender reports streaming.** The phone advertises several SPP services, some belonging to the system. The Linux sender filters to the service named **`PPGMonitor`**. If duplicate PPGMonitor records linger from earlier runs, cycling the phone's Bluetooth off and on while the app is closed, then reopening the app once, registers a single clean record.
- **`INSTALL\_FAILED\_UPDATE\_INCOMPATIBLE`.** An existing copy is signed with a different key. Uninstall it first (section 4).
- **HR and RR stay "--" indefinitely.** Most often a sample-rate mismatch — see section 11. Insufficient elapsed time is the other cause; see section 9.

- **A blank screen.** The phone must be unlocked and the app in the foreground.

Appendix A — Command reference

A.1 Windows

```
pip install pyserial
```

```
python send_bt_win.py COM3 ..\synthesized_sample_adult.txt 125 --subject adult
```

```
python send_bt_win.py COM3 ..\synthesized_sample_neonate.txt 125 --subject neonate
```

Confirm the COM port, from PowerShell:

```
[System.IO.Ports.SerialPort]::GetPortNames()
```

Open the Bluetooth COM-port dialog, from **Win + R**:

```
rundll32.exe shell32.dll,Control_RunDLL bthprops.cpl,,1
```

A.2 Linux

```
sudo apt-get install -y libbluetooth-dev
```

```
pip install pybluez2
```

```
python3 send_bt.py xx:xx:xx:xx:xx:xx ..\synthesized_sample_adult.txt 125 --subject adult
```

```
python3 send_bt.py xx:xx:xx:xx:xx:xx ..\synthesized_sample_neonate.txt 125 --subject neonate
```

A.3 Installing the app, if adb is available

```
adb install -r -t app-debug.apk
```

```
adb uninstall com.example.ppgmonitor
```